

**LAPORAN SIMULASI CACHING PADA WEBSITE
TOP UNIVERSITY IN BANGKOK MENGGUNAKAN
PYTHON FLASK**



OLEH :

NAMA : AL - HIZRA
NIM : 105841101523
KELAS : RPL - 6.A

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH MAKASSAR
2026**

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai bentuk implementasi materi Scalable System Design khususnya pada topik caching.

Dalam laporan ini dilakukan simulasi penerapan caching menggunakan Python Flask dengan strategi Cache Aside Pattern. Simulasi dibuat dalam bentuk website sederhana yang menampilkan informasi universitas terkemuka di Bangkok. Melalui simulasi ini dapat diamati bagaimana caching mampu mempercepat waktu respons sistem dan mengurangi akses langsung ke database.

Saya menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa mendatang.

Narathiwat, 11 Jun 2026

Al Hizra

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Rumusan Masalah	1
1.3 Tujuan.....	1
1.4 Manfaat.....	1
1.5 Batasan Masalah.....	2
BAB II LANDASAN TEORI	3
2.1 Pengertian Caching.....	3
2.2 Konsep Dasar Cache Hit dan Cache Miss	3
2.3 Strategi Cache Aside Pattern	4
2.4 Peran Caching dalam Scalable System.....	5
BAB III METODE DAN IMPLEMENTASI.....	6
3.1 Metode Penelitian.....	6
3.2 Implementasi Sistem	6
BAB IV HASIL DAN PEMBAHASAN	7
4.1 Hasil Pengujian Simulasi.....	7
4.2 Pembahasan Efisiensi Performa	9
BAB V PENUTUP.....	10
5.1 Kesimpulan.....	10
5.2 Saran	10
DAFTAR PISTAKA	11

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi menyebabkan jumlah pengguna aplikasi dan website terus meningkat. Semakin banyak pengguna yang mengakses sistem secara bersamaan, semakin besar pula beban yang harus ditangani oleh server dan database.

Pada banyak aplikasi, data yang sama sering diminta berulang kali oleh pengguna. Jika setiap permintaan harus mengambil data langsung dari database, maka akan terjadi peningkatan waktu respons dan penggunaan sumber daya yang tidak efisien.

Salah satu solusi yang digunakan dalam scalable systems adalah caching. Caching memungkinkan data yang sering diakses disimpan sementara pada media yang lebih cepat sehingga permintaan berikutnya dapat diproses tanpa harus mengakses database kembali.

Untuk memahami konsep tersebut, dilakukan simulasi penerapan caching menggunakan Python Flask pada website informasi universitas terkemuka di Bangkok dengan menerapkan strategi Cache Aside Pattern.

1.2 Rumusan Masalah

1. Apa yang dimaksud dengan caching dalam scalable system?
2. Bagaimana konsep Cache Hit dan Cache Miss bekerja?
3. Bagaimana implementasi Cache Aside Pattern pada aplikasi berbasis Python Flask?
4. Bagaimana pengaruh caching terhadap waktu respons aplikasi?

1.3 Tujuan

1. Memahami konsep dasar caching
2. Memahami mekanisme Cache Hit dan Cache Miss
3. Mengimplementasikan Cache Aside Pattern menggunakan Python Flask
4. Menganalisis pengaruh caching terhadap performa aplikasi

1.4 Manfaat

Simulasi caching ini bermanfaat untuk meningkatkan pemahaman mahasiswa mengenai konsep scalable systems dan implementasi caching secara praktis. Selain itu, simulasi ini membantu pengembang memahami peran caching dalam meningkatkan

performa aplikasi serta dapat menjadi referensi dasar bagi pembaca dalam mempelajari teknologi caching.

1.5 Batasan Masalah

Penelitian ini berfokus pada simulasi caching menggunakan Python Flask dengan strategi Cache Aside Pattern. Database dan cache yang digunakan berupa dictionary Python sebagai simulasi penyimpanan data. Data yang digunakan adalah informasi universitas di Bangkok, dan pengujian dilakukan pada lingkungan localhost untuk mengamati perbedaan waktu respons antara Cache Hit dan Cache Miss.

BAB II

LANDASAN TEORI

2.1 Pengertian Caching

Caching adalah teknik penyimpanan sementara data yang sering digunakan pada media yang memiliki kecepatan akses lebih tinggi dibandingkan sumber data utama. Tujuan utama caching adalah mengurangi waktu respons aplikasi serta mengurangi beban server dan database. Ketika data yang sama diminta kembali, sistem dapat mengambil data langsung dari cache tanpa perlu melakukan proses pengambilan ulang dari database.

2.2 Konsep Dasar Cache Hit dan Cache Miss

1. Cache Hit

Cache Hit terjadi ketika data yang diminta pengguna tersedia di dalam cache.

Prosesnya:

```
User
  ↓
Cache
  ↓
User
```

Keuntungan:

- a. Waktu respons lebih cepat
- b. Beban database berkurang
- c. Pengalaman pengguna meningkat

2. Cache Miss

Cache Miss terjadi ketika data yang diminta belum tersedia dalam cache.

Prosesnya:

```
User
  ↓
Cache
  ↓
Database
  ↓
Cache
  ↓
User
```

Pada kondisi ini sistem harus mengambil data dari database terlebih dahulu kemudian menyimpan ke cache untuk digunakan pada permintaan berikutnya.

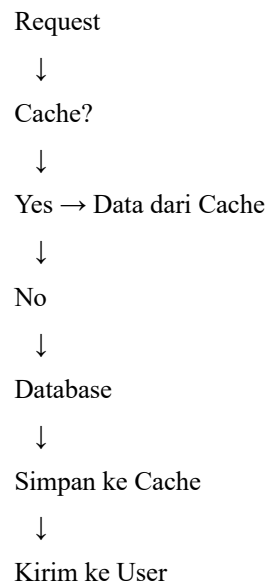
2.3 Strategi Cache Aside Pattern

Cache Aside Pattern merupakan strategi caching yang paling banyak digunakan dalam aplikasi modern.

Alur kerjanya sebagai berikut:

1. Aplikasi memeriksa cache terlebih dahulu
2. Jika data tersedia, data langsung digunakan
3. Jika data tidak tersedia, aplikasi mengambil data dari database
4. Data kemudian disimpan ke cache
5. Permintaan berikutnya akan mengambil data dari cache.

Diagram Proses:



Kelebihan:

- a. Mudah diimplementasikan
- b. Mengurangi beban database
- c. Cocok untuk data yang sering dibaca

Kekurangan:

- a. Data cache dapat menjadi usang apabila database berubah

2.4 Peran Caching dalam Scalable System

Pada scalable systems, caching berfungsi sebagai lapisan tambahan yang membantu mengurangi jumlah akses langsung ke database. Dengan menyimpan data yang sering digunakan pada media penyimpanan yang lebih cepat, sistem dapat memberikan respons yang lebih singkat kepada pengguna tanpa harus selalu mengambil data dari sumber utama. Penerapan caching juga dapat mengurangi jumlah query ke database, menekan penggunaan bandwidth, serta mengurangi beban kerja server. Selain itu, caching memungkinkan sistem untuk melayani lebih banyak permintaan pengguna secara bersamaan sehingga meningkatkan skalabilitas dan efisiensi sistem secara keseluruhan tanpa memerlukan peningkatan sumber daya yang signifikan.

BAB III

METODE DAN IMPLEMENTASI

3.1 Metode Penelitian

Metode yang digunakan adalah simulasi implementasi caching menggunakan Python Flask.

Tahapan penelitian:

1. Mempersiapkan data universitas
2. Membuat simulasi database
3. Membuat cache menggunakan dictionary Python
4. Mengimplementasikan Cache Aside Pattern.
5. Mengukur waktu respons sistem
6. Menganalisis hasil Cache Hit dan Cache Miss.

3.2 Implementasi sistem

Sistem dibangun menggunakan:

Komponen	Teknologi
Bahasa Pemrograman	Python
Framework	Flask
Front-End	HTML
Cache	Python Dictionary
Database	Simulasi Dictionary

Data yang digunakan:

1. Chulalongkorn University
2. Mahidol University
3. Thammasat University

Pada saat pengguna memilih universitas, aplikasi akan memeriksa cache terlebih dahulu.

Jika data tersedia maka sistem menampilkan status Cache Hit. Jika data belum tersedia maka sistem mengambil data dari database simulasi dan menampilkan status Cache Miss.

BAB IV

HASIL DAN PEMBAHASAN

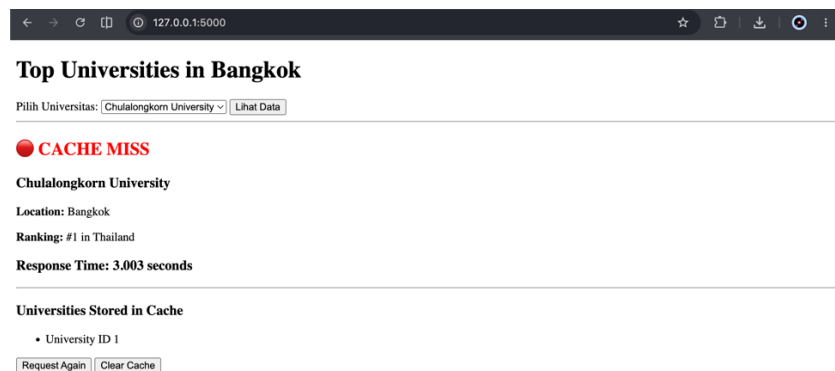
4.1 Hasil Pengujian Simulasi

1. Pengujian Pertama:

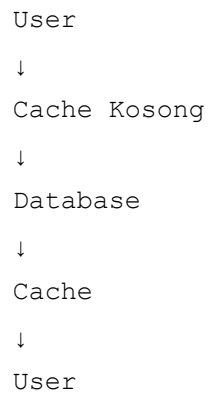
a. Kondisi:

- Cache kosong
- Pengguna membuka data Chulalongkorn University

b. Hasil:



c. Proses:

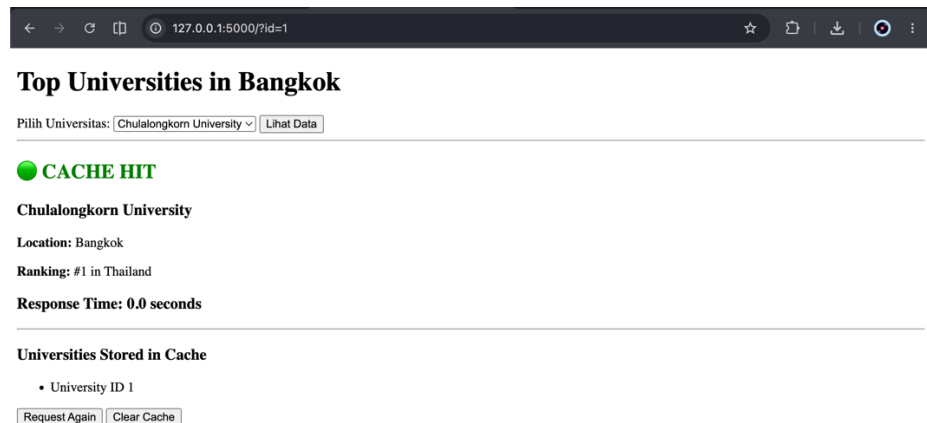


2. Pengujian Kedua:

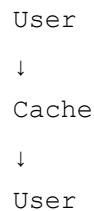
a. Kondisi:

- Data sudah tersimpan di cache
- Pengguna membuka data yang sama

b. Hasil:



c. Proses:

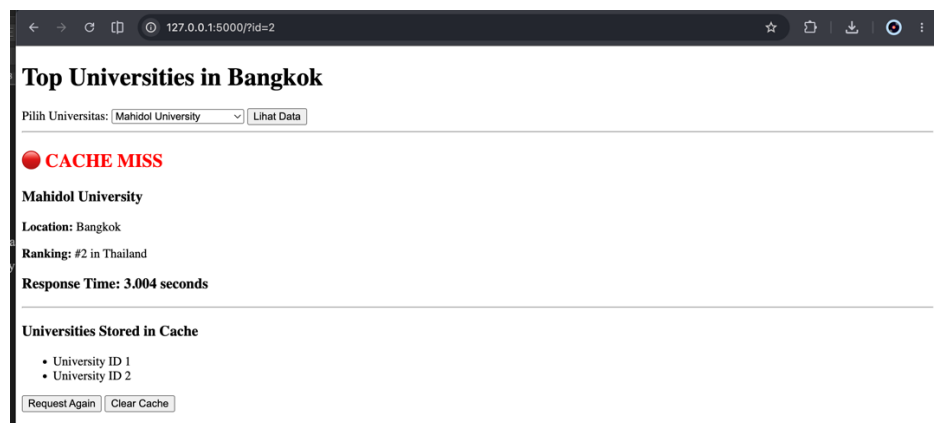


3. Pengujian Ketiga

a. Kondisi:

- Pengguna membuka Mahidol University untuk pertama kali

b. Hasil:

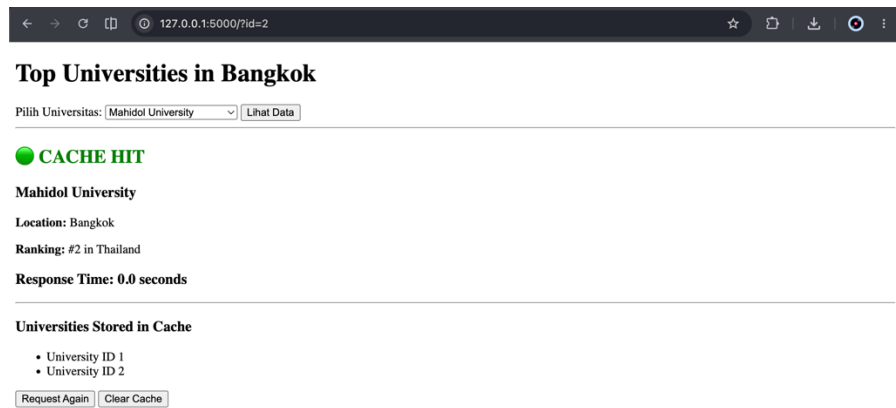


4. Pengujian Keempat

a. Kondisi:

- Pengguna membuka Mahidol University kembali

b. Hasil:



4.2 Pembahasan Efisiensi Performa

Berdasarkan hasil pengujian, terlihat bahwa caching memberikan peningkatan performa yang signifikan.

Perbandingan:

Kondisi	Waktu respons
Cache Miss	± 3 detik
Cache Hit	± 0 detik

Pengurangan waktu respons mencapai hampir 100%.

Hal ini menunjukkan bahwa penggunaan cache mampu mengurangi akses database dan mempercepat proses penyajian data kepada pengguna.

Dalam skala besar, teknik ini sangat penting karena dapat membantu sistem menangani ribuan hingga jutaan permintaan secara lebih efisien.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan hasil simulasi yang telah dilakukan, dapat disimpulkan bahwa caching merupakan teknik yang efektif untuk meningkatkan performa aplikasi.

Implementasi Cache Aside Pattern pada aplikasi Python Flask berhasil menunjukkan perbedaan antara Cache Hit dan Cache Miss. Pada kondisi Cache Miss, sistem memerlukan waktu sekitar 3 detik karena harus mengakses database. Sebaliknya, pada kondisi Cache Hit, data dapat diambil langsung dari cache sehingga waktu respons menjadi hampir 0 detik.

Dengan demikian, caching terbukti mampu mengurangi beban database, mempercepat waktu respons, dan mendukung pembangunan scalable systems yang lebih efisien.

5.2 Saran

1. Menggunakan Redis atau Memcached sebagai cache pada implementasi nyata.
2. Menambahkan mekanisme cache expiration atau TTL.
3. Mengembangkan sistem dengan jumlah data yang lebih besar.
4. Mengimplementasikan strategi caching lainnya seperti Write Through atau Read Through untuk perbandingan performa.

DAFTAR PUSTAKA

- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- Richardson, C. (2018). *Microservices Patterns*. Manning Publications.
- Nygard, M. (2018). *Release It! Design and Deploy Production-Ready Software*. Pragmatic Bookshelf.
- Redis Documentation. Available at: <https://redis.io/docs/> .
- Memcached Documentation. Available at: <https://memcached.org/> .
- Flask Documentation. Available at: <https://flask.palletsprojects.com/> .
- Microsoft Azure Architecture Center. *Cache-Aside Pattern*. Available at: <https://learn.microsoft.com/en-us/azure/architecture/patterns/cache-aside>.